《FPGA 工具与基础设计》

《FPGA 工具与基础设计》 1
一：仿真工具（Modelsim） 3
1.1 仿真库 3
1.2 仿真分类 4
1.2.1 独立仿真 5
1.2.2 自动化仿真 9
1.2.3 联合仿真 10
二：HDL 设计基础 12
2.1 HDL 建模 13
2.1 数据流建模 14
2.2 行为建模 15
2.2.1 组合电路建模 16
2.2.2 时序电路建模 17
2.2.3 关于组合电路和时序时序特征 19
2.3 结构化建模 20
2.3.1 结构化建模设计案例 21
三：基础电路设计 23
3.1 触发器 23
3.2 多路器 24
3.3 译码器 26
3.4 计数器 27

一：仿真工具（Modelsim）

仿真是 HDL 设计的检查电路设计是否符合我们预期的一种手段，非常高效且使用，其属于验证学的一个分支，属于动态验证的。仿真需要结合仿真器，通过激励文件的激励信号值，观测响应信号的变化。

仿真的原理就是通过对电路系统的激励信号作用，观测响应信号的变化是否正确，通过虚拟仿真器进行动态模拟，数字电路设计中常用的仿真器有：

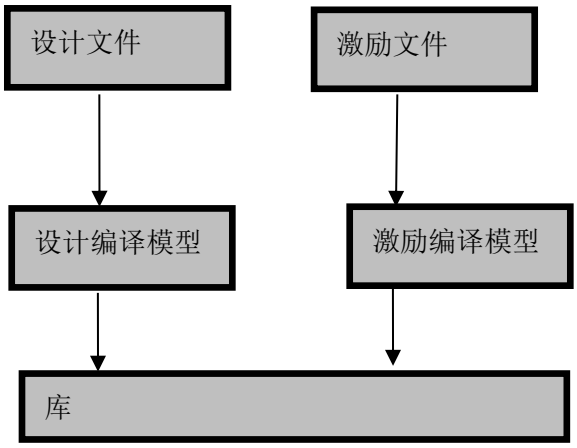
EDA 仿真器	应用目标	操作系统支持	HDL 语言支持
Modelsim	FPGA/CPLD(半定制开发)	windows	verilog/vhdl
Questasim	验证	windows/linux	system verilog system C
vcs	ASIC（全定制开发）	linux	system verilog system C verilog/vhdl

在 FPGA 开发中主要以 Modelsim 作为仿真器的标杆，对应不同的场景有不同的版本，根据自己情况下载适合自己的版本。

免费版：完全免费，比如：modelsim-altera
教育版：完全免费
专业版：收费，可破解
企业版：收费

1.1 仿真库

任何仿真都需要将设计输入文件经过编译，生成编译模型（仿真编译文件），这些编译模型需要存放在库中，所谓的库就是一个存储空间虚拟映射，这段存储空间的物理映射名叫做库。



仿真库原理示意图

Note:任何设计输入文件都必须经过编译为仿真模型才能进行仿真

1.2 仿真分类

在数字电路设计中，通常仿真分为 3 钟，每个仿真都有自己的优缺点，实际中我们根据需要的情况选择适合自己的仿真方式。

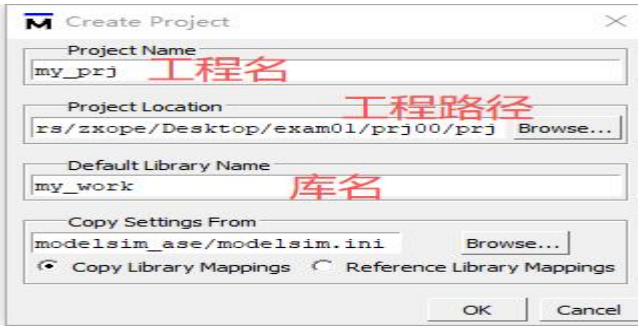
仿真方式	描述	特点
独立仿真	不依赖于任何其他工具，只需要仿真	优点： 不需要其他软件参与，节省仿真时间 缺点： 有的必要的仿真模型需要手动编译

	器即可完成仿真。	
联合仿真	仿真器 + 其他工具	优点:通常通过集成开发环境直接调用仿真器，所有的仿真库比编译在后天自动完成，对新手友好 缺点：需要同时掌握多个工具
自动化仿真	通过脚本命令完成自动化仿真	通过手动编写脚本完成自动仿真的一种仿真形式，比如：modelsim 的脚本为 do 脚本

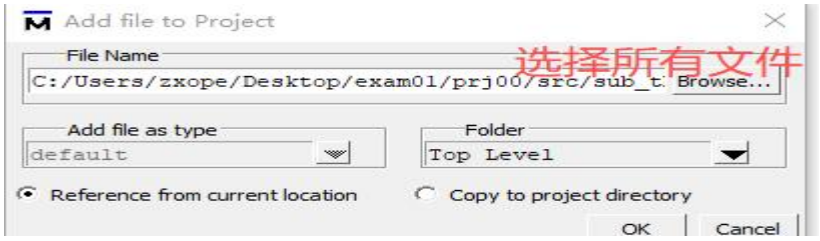
1.2.1 独立仿真

第 01 步：完成设计文件和激励文件的编写

第 02 步: 新建仿真工程



第 03 步: 添加所有参与仿真的文件

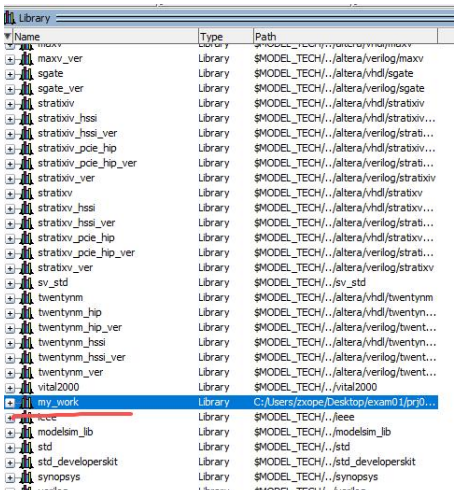


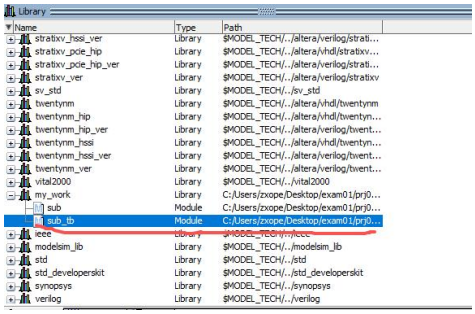
第 04 步: 编译所有文件

编译所有文件，如果有语法错误，则修改代码，没有的话编译后的对应的所有的编译模型自动保存在当前工程所对应的库中，比如：my_work

第 05 步: 启动仿真

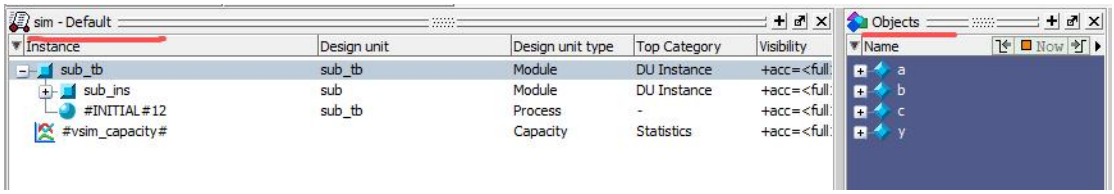
找到当前工程对应的库，比如 my_work，找到 tesetbench 的编译模型，启动仿真。





第 06 步: 添加需要观测信号到波形显示窗口

找到仿真模型管理窗口（比如：modelsim-sim）选中需要添加信号的编译模型，在目标窗口（modelsim-object)会显示你选中的编译模型所有的信号，直接在目标窗口选中我们需要观测的信号添加即可，反复操作。

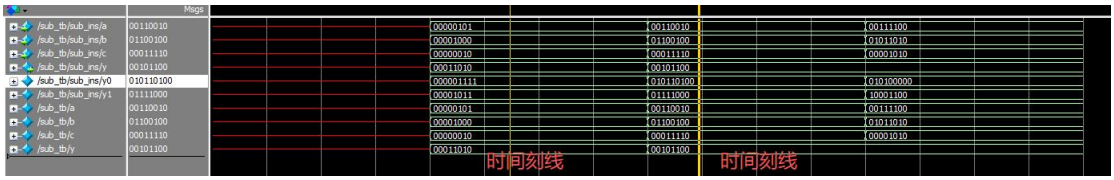


第 07 步: 运行仿真

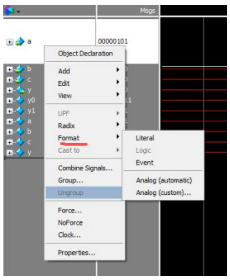
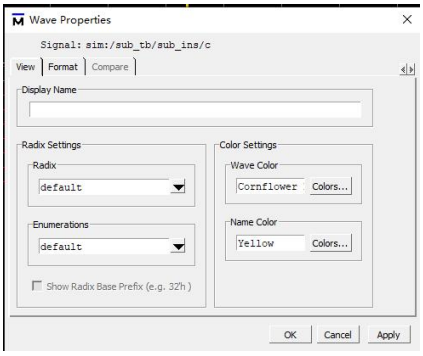
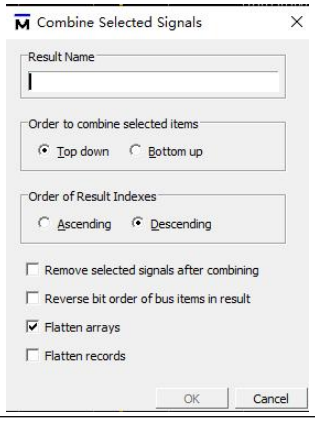
运行操作	描述
重新运行	重新仿真
单步运行	需要设定运行时长，仿真按照我们设定的运行时长进行仿真
连续运行	需要抖动停止运行

第 08 步: 波形分析

在观测波形的时候以当前时刻为基准，所谓当前时刻为我们手动选择的时间基线，通常通过 mark 表示，在实际中当前时刻刻选择多个参考点，通过添加，产出时间刻线选择当前时刻的参考点。



波形分析常用的操作	目的
放大	波形放大
缩小	波形缩小
全屏显示	波形全屏显示
时间测量	添加/删除时间刻线
进制选择	设置信号值的显示格式

信号沿跳变	
信号简单显示	
离散连续信号转换	
波形形态设置	设置波形形态 
信号合并	多个信号可以合并为一个信号 

Note:不管哪种仿真，所有的波形分析一样。

1.2.2 自动化仿真

数字电路设计中一种只能编写的仿真形式叫做自动化仿真，通过脚本命令完成仿真，modelsim 的脚本为 do 脚本。

第 1 步：创建脚本文件

my_sim.do

第 2 步：定位工作路径

```
cd     C:/Users/zxope/Desktop/exam01/prj01/sim
```

第 3 步：定义虚拟库

```
vlib  my_work;
```

第 4 步：虚拟库映射

```
vmap  my_work     my_work;
```

第 5 步：编译所有与仿真有关的文件

```
vlog  -work  my_work    ../src/*.v
```

第 6 步：启动仿真

```
vsim  my_work.sub_tb
```

第 7 步：打开波形显示窗口

```
view   wave  ;
```

第 8 步：删除所有波形

```
delete wave  /*;
```

第 9 步：添加需要的波形

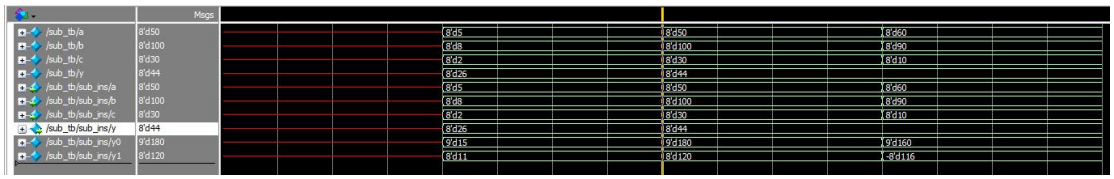
```
add  wave  -dec   /sub_tb/*  ;
add  wave  -dec   /sub/*  ;
```

第 10 步：运行仿真

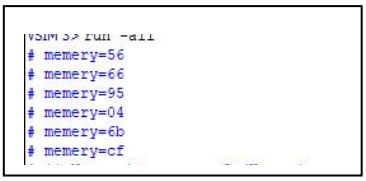
```
run  100ns;
```

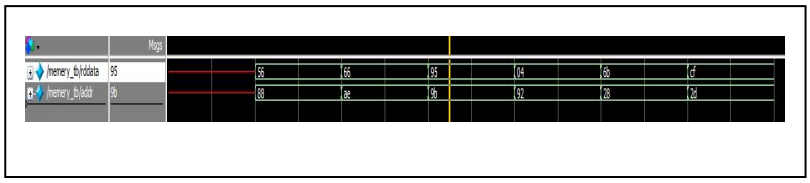
第 11 步：启动仿真器

```
cd   C:\Users\zxope\Desktop\exam01\prj01\sim
vsim  -do   "my_sim.do"
```



仿真案例
标题:存储初始化仿真
要求：
1：reg [7:0] mem [255:0]
2: $display 打印存储器的数据到终端
3：通过 for 循环初始化 mem
4：独立仿真和自动化仿真进行验证





1.2.3 联合仿真

联合仿真是仿真方式之一，无论是新手还是经验丰富的开发者都非常实用，80%开发者都喜欢选择这种方式。

所谓联合仿真就是仿真器结合其他 EDA 工具（通常为集成开发环境）完成仿真操作，通常为集成开发环境+仿真器使用较多。

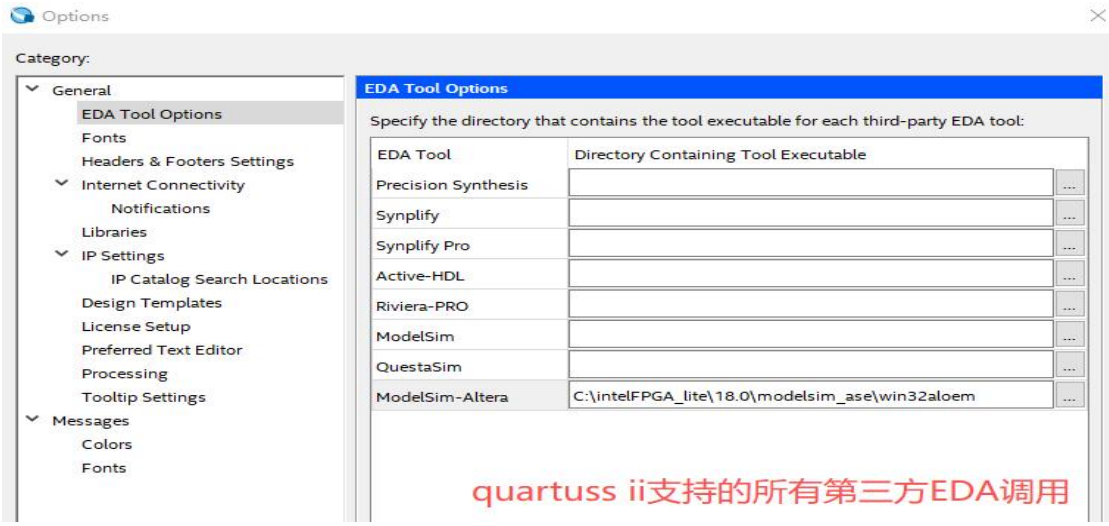
如果是集成开发环境+仿真器的联合仿真，一般为集成开发环境调用仿真器。

联合仿真不需要单独编译我们在仿真中必须使用的额外仿真库，独立仿真和自动化仿真都需要，所以他比较友好。

对于 FPGA 开发者联合仿真基本为集成开发环境+仿真器的形式，我们这里重点介绍这种仿真的操作流程。

第 1 步：完成设计文件和激励文件的编写
第 2 步：在集成开发环境中设置仿真器的安装路径

由于集成开发环境需要调用仿真器，因此必须指定安装路径，便于集成开发环境准确找到仿真器，才能调用。

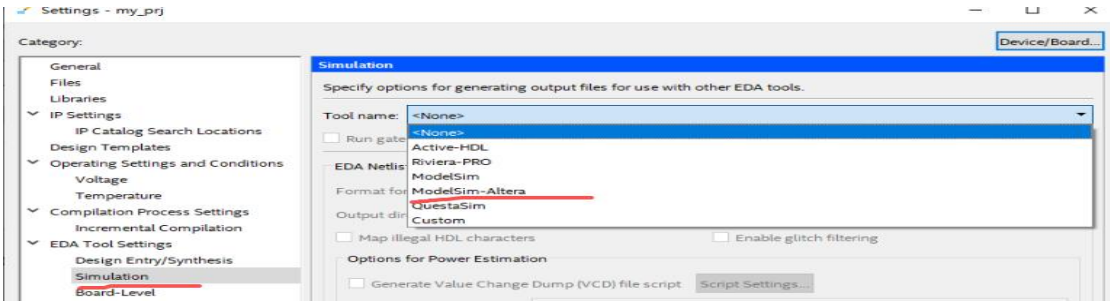


第 3 步：如果有必要需要设置环境变量

有的操作系统在安装 modelsim 的时候没有自动完成环境变量的修改，需要手动修改，如果环境变量不正常是无法正常调用。

第 4 步：设置当前使用的仿真器

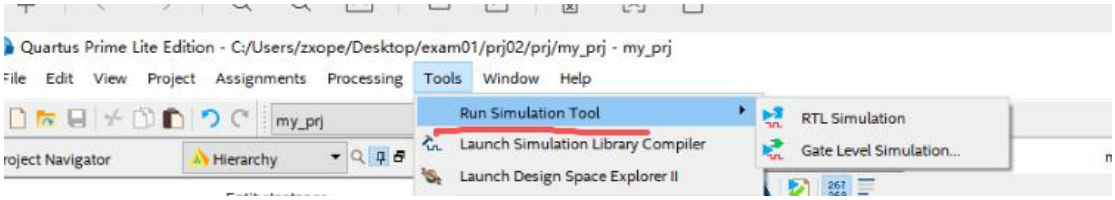
由于仿真器不只有 modelsim 一种，因此我们需要指定使用哪种。



第 5 步：设置 testbench;

任何仿真必须指定才能正常仿真。

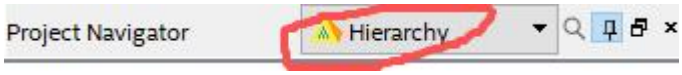
第 6 步：启动仿真

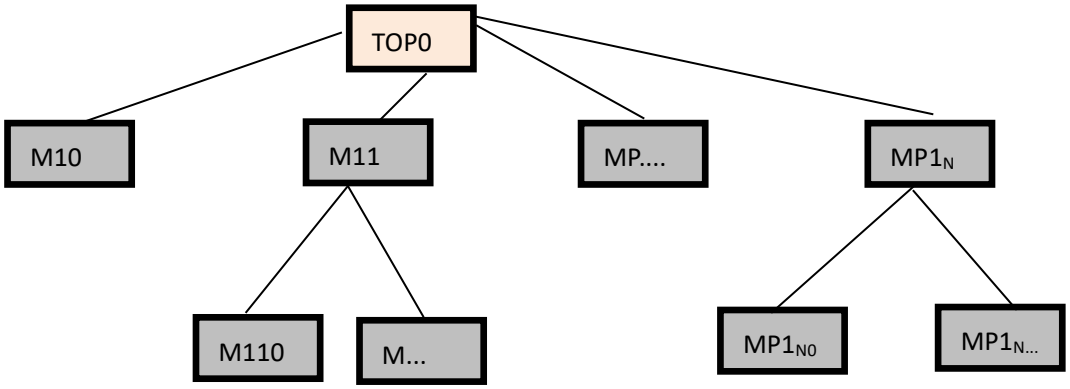


二：HDL 设计基础

HDL 是数子电路设计的一种高效使用的设计方式，任何电路设计以模块为基本单元，所谓的模块就是能够独立完成某一功能的电路，在电路设计中要白痴模块的独立性，从电路系统角度来看，模块本身就是一个电路系统。

在 HDL 设计中，小的电路系统构成大的电路系统，这种特性天然形成一种层次化的结构，因此任何电路具有层次化，在设计中，我们就像搭积木一样，由于电路设计这种属性从而衍生出顶层模块。





电路设计核心要素：

模块核心要素	描述
层次化	电路设计的自然属性
顶层模块	电路设计的自然属性诞衍生出的一种模块，对于一个完整的电路只能有一个顶层模块。顶层模块对于综合器而言为顶层实体。
端口	模块之间通信的桥梁
可配置	模块通用性一种设计方法
建模	HDL 设计中流行的一种概念，通过 HDL 构建电路的模型。

2.1 HDL 建模

在 HDL 设计中广为流程的一种设计思想，是构建任何电路的基本思想，这种设计思想用来描述模块行为运行的特征的设计方法。

如：

```
module   add   #(//模块配置
                parameter   WIDTH   =   8

)
(//模块的端口
                input   [WIDTH-1:0] a,
                input   [WIDTH-1:0] b,
                output   [WIDTH-1:0] c,



);

功能设计（行为特征描述）;

endmodule
```

广为流传的建模方式有 3 种方式，分别数数据流建模，行为建模，结构化建模，任

何电路模块描述功能都离不开这三种方式。

模块功能描述设计方式	描述
数据流建模	简单表示驱动信号，负载信号，变化，条件信号之间的一个逻辑关系，只能用来设计组合电路 每一个 HDL 语言都会提供这样的建模方式。
行为建模	通用的建模方式，相比较数据流建模，这种方式用来描述更复杂的驱动信号，负载信号，变化，条件信号之间的一个逻辑关系，组合电路和时序电路都可以设计
结构化建模	HDL 设计中模块层次化建模的直观体型

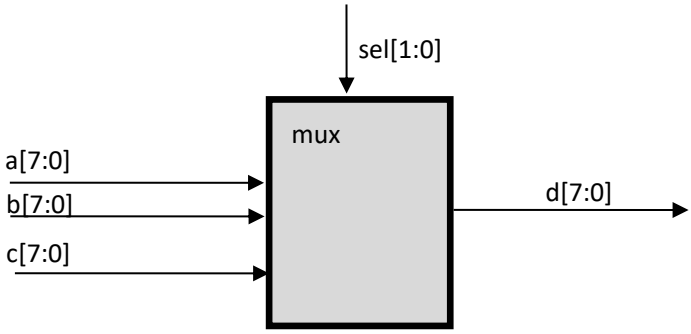
2.1 数据流建模

简单表示驱动信号，负载信号，变化，条件信号之间的一个逻辑关系，只能用来设计组合电路，每一个 HDL 语言都会提供这样的建模方式，并且提供了这样的行为运行特征描述语句。

语言	行为特征运行描述关键字
verilog	**assign**
vhdl	**wen** **with** **直接赋值**

设计案例：3 选 1 多路器
要求：
1：组合电路
2：通过数据流建模
3：真值表

a[7:0]	b[7:0]	c[7:0]	sel[1:0]	d
a	b	c	0	a
a	b	c	1	b
a	b	c	2/3	c




```
module          mux
(
            input   wire    [7:0]   a,
            input   wire    [7:0]   b,
            input   wire    [7:0]   c,
            output wire     [7:0]   d
);
//数据流建模，用来描述 mux3 模块的功能
assign          d       =   (sel==2'd0)         ?           a[7:0]
    :
                    (sel==2'd1)         ?           b[7:0]          :
                                                    c[7:0]          ;


endmodule
```

2.2 行为建模

行为建模是任何 HDL 语言的一种建模方式，提供给我门一种相对于数据流建模更为通用，且复杂的一种建模方式，任何数据流建模设计的电路，都可以用行为建模来设计。

通过这种建模，我们可以描述任何电路行为，简单的，复杂的都可以，既可以设计组合电路又可以设计时序电路。

在 HDL 语言中，通过行为运行特征描述中的进程控制语句，简称进程语句。

语言	行为特征运行描述关键字
verilog	**always** **initial**
vhdl	**process**

2.2.1 组合电路建模

在 verilog 中通过 **always** 进行组合电路设计，具体如下;

```
always        @(敏感信号 0，敏感信号 1,.....)
always        @(*)
always        @*
```

敏感信号列表

条件控制符

运行行为特征描述关键字

设计案例：3 选 1 多路器

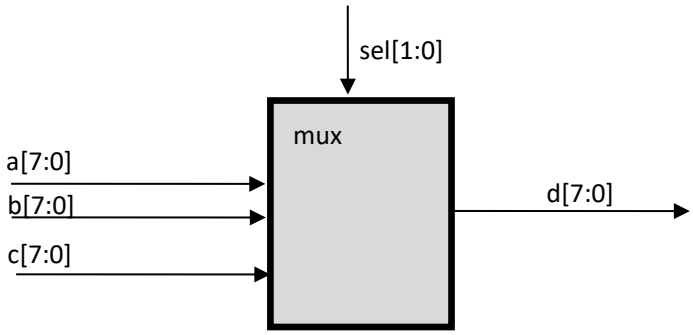
要求：

1：组合电路

2：通过数据流建模

3：真值表

a[7:0]	b[7:0]	c[7:0]	sel[1:0]	d
a	b	c	0	a
a	b	c	1	b
a	b	c	2/3	c

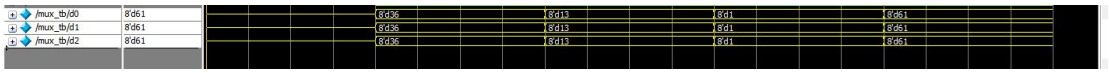



```
module          mux
    (
                    input   wire    [7:0]   a,
                    input   wire    [7:0]   b,
                    input   wire    [7:0]   c,
```

```
                    input wire    [1:0]    sel,
                    output reg     [7:0]    d
    );
    //行为建模，用来描述 mux3 模块的功能
always                    @*
                    if(sle==0)
                         d=a;
                    else if(sel==1)
                         d=b;
                    else
                         d=c;



    endmodule
```



2.2.2 时序电路建模

时序电路就是将组合电路的信号进行采样，并寄存，时序电路设计中有 2 个关键要点：

1：采样

采样方式	描述
沿采样	在采样时钟的边沿采样
电平采样	在采样时钟的稳定的电平阶段采样

Note:在 HDL 设计中时序电路单只边沿采样。

可以看出任何时序电路都必须有采样时钟，因此时序电路建模必须有时钟信号。

2：寄存

当采样到数据后，将其值保存在寄存器，但是第一个采样之前寄存器需要初始化，在时序电路设计中通过复位信号进行初始化，因此时序电路建模有一个关键信号就是复位信号。

综上所述，在任何时序电路建模的过程中涉及到 2 个关键信号，时钟和复位。

在 verilog 中通过 **always** 进行组合电路设计，具体如下：

```
always      @（posedge   clk ）
always      @（negedge   clk ）
always      @（negedge   clk        ）
            if(reset==1)
                ...;
```

```
            else
                ...;
always      @（negedge   clk      ）
            if(reset==0)
                ...;
            else
                ...;

always      @（negedge   clk      or   posedge   reset ）
                if(reset==1)
                ...;
            else
                ...;
always      @（negedge   clk      or   negedge   rst_n ）
                ifrst_n==0)
                ...;
            else
                ...;
```

设计案例：3 选 1 多路器

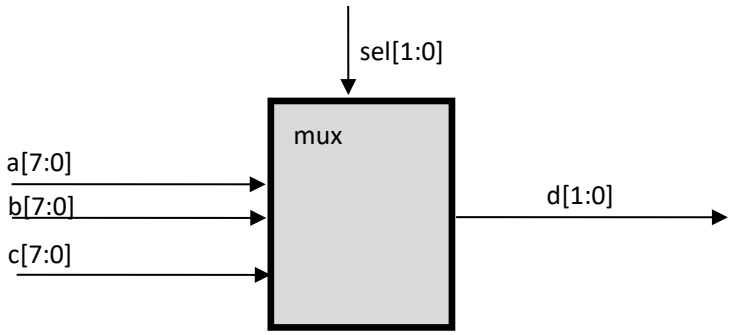
要求：

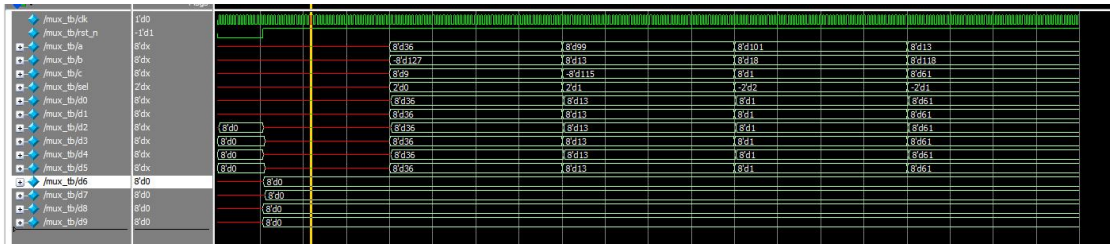
1：时序电路

2：通过行为建模

3：真值表

a[7:0]	b[7:0]	c[7:0]	sel[1:0]	d
a	b	c	0	a
a	b	c	1	b
a	b	c	2/3	c





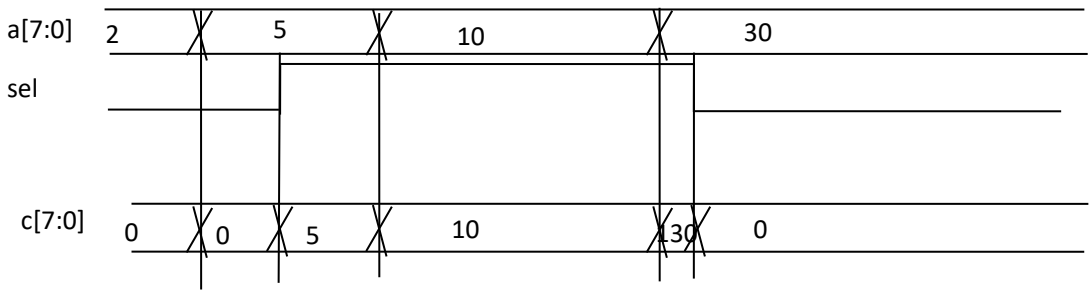
2.2.3 关于组合电路和时序时序特征

在数字电路设计中，组合电路和时序的时序特性存在显著不同，对于我们刚开始接触数字电路设计特别容易混淆的 2 个特征。

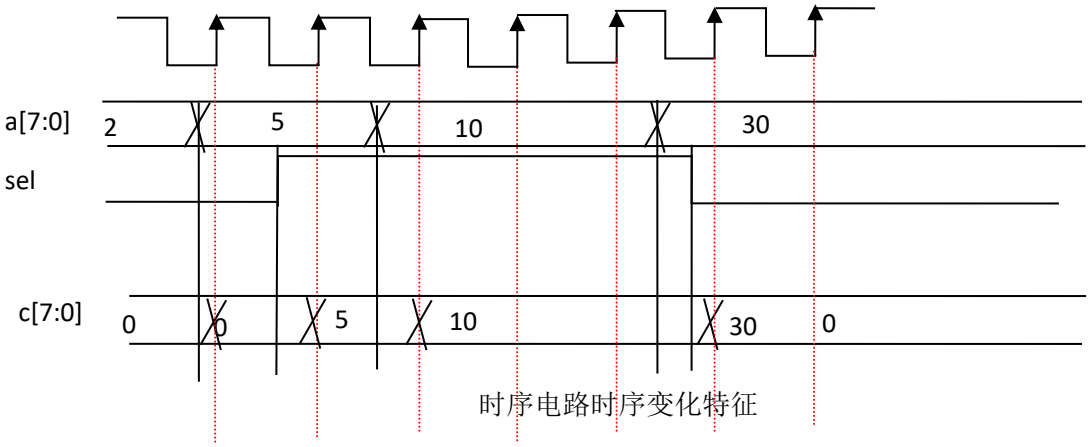
组合电路：理论上，当敏感信号的值发生变化的时候，负载信号的值立即响应。
时序电路：理论上，当敏感信号的值发生变化的时候，负载信号的值不是立即响应，而是在采样时刻发生变化。

```
always    @(*)
          if(sel)
               c= a;
          else
               c=0;
```

```
always    @(posedge          clk)
          if(sel)
               c<= a;
          else
               c<=0;
```



组合电路时序变化特征



时序电路时序变化特征

NOTE：对应仿真器的功能仿真的时序都是模拟理论上的时序。

以上是基本的时序特征，在数字电路设计中我们还需要关注复位信号，复位信号是时序电路设计提供上电时刻寄存器初始化的一种方式。

在电路设计中复位信号需要满足以下要求;

1:复位时钟至少大于电路设计中最慢的时钟的一个时钟周期。
如：电路中最慢的时钟 1khz,则复位时钟必须大于 1ms
2:复位信号的电平变化不能发生在电路设计中任何时钟的时钟沿。
3：复位有异步和同步，在实际这种正确选择复位方式

复位方式	特点	使用场景
异步复位	复位和时钟没关系 真正意义上复位	首选异步复位，但是需要注意复位信号是否满足条件，如果我们不能确定，通常采用同步释放的方法。 有的 IP 不支持异步复位的情况下，必须用同步复位
同步复位	复位必须与时钟同步 本质是寄存器的数据选择	同步复位某种意义上即使在复位信号不理想的情况下，试错率高，但是使用设计资源。 有 IP 的必须使用同步复位

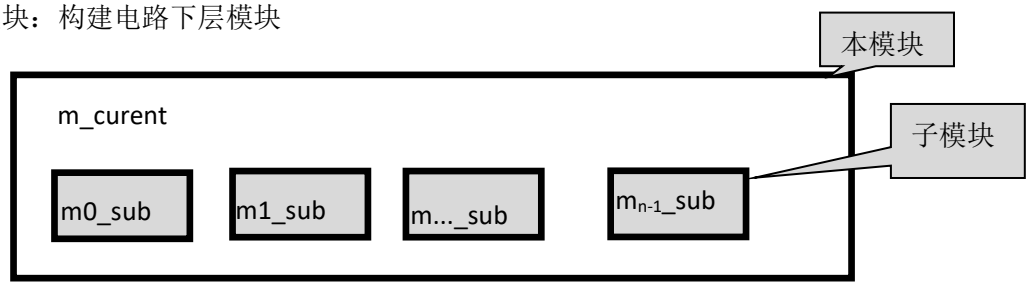
4：复位信号不能有毛刺

2.3 结构化建模

电路模块具备层次化的自然属性，在电路设计中，必须有针对这种属性专用的建模方式，即结构化建模，任何 HDL 语言都必须支持结构化建模。

结构化建模针对的是各个模块之间的层次化的属性，本质就是搭积木，在接口化建模总核心为模块，包括;

- 本模块：构建电路的当前模块（上层模块）
- 子模块：构建电路下层模块



在 HDL 设计中，本模块中的子模块需要以原件的方式存在，换句话说子模块在本模块中我们称之为元件，在本模块中集成子模块对应元件的过程我们通常叫做例化。

元件：本模块中子模块的一种存在形态
例化：原件集成在本模块中的一个过程

在 HDL 设计中，根据结构化建模的建模方式的特点，**标准的例化**需要通过 2 个操作:

- 定义原件

  在结构化建模中，子模块需要先定义为元件。

- 端口映射

  元件作为模块，需要通过端口传递信息，因此元件的端口必须要与本模块的信号进行绑定才能传递信息。

在不同的 HDL 语言总例化操作与不一样，在 verilog 中采用的一种简化的例化方式，vhdl 采用标准的例化方式。

```
模块名     元件名     //元件声明
//端口映射
（
端口映射

）；

如   ：add   add_ins//元件声明
        （


        ）；
```

在 verilog 例化的端口映射有 2 种：
1：顺序映射
2：端口名映射

2.3.1 结构化建模设计案例

标题：异或门，同或门电路 LUT 构建
要求：
1：通过与门，或者，非门，构建异或门
2：与门，或门，非门，通过 Lut 构建
3：通过独立仿真，自动化仿真验证

异或门真值表

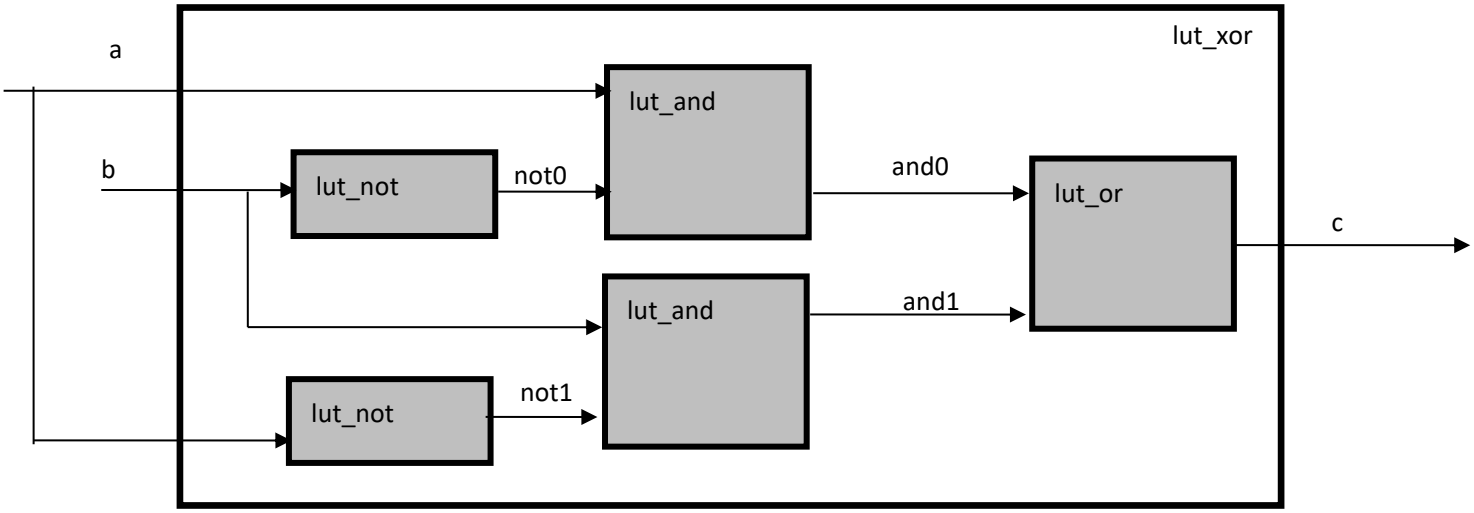
a	b	c
0	0	0
0	1	1
1	0	1
1	1	0

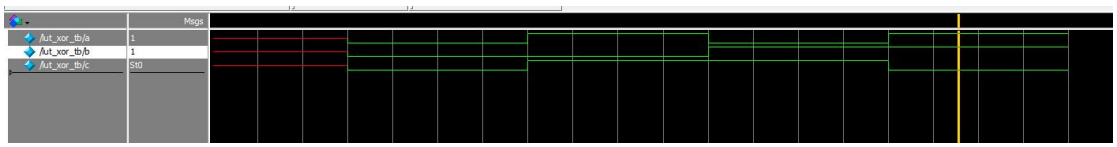
同或门真值表

a	b	c
0	0	1
0	1	0
1	0	0
1	1	1

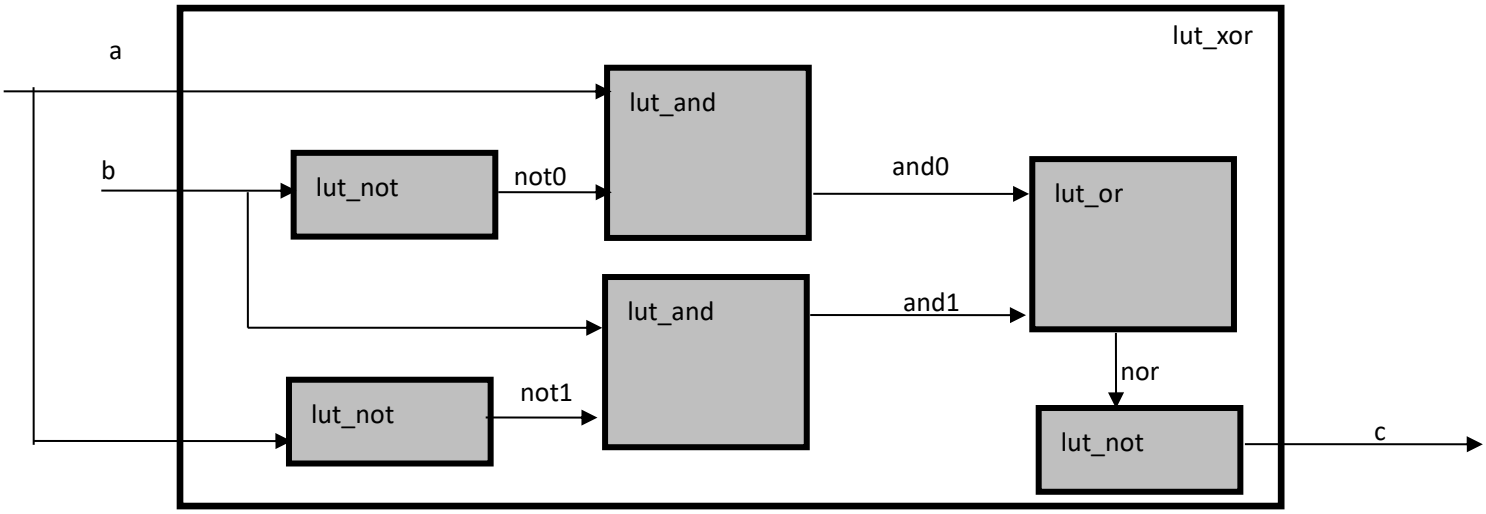
目前公认的异或门的逻辑表示如下：

$c = a\bar{b} + \bar{a}b$

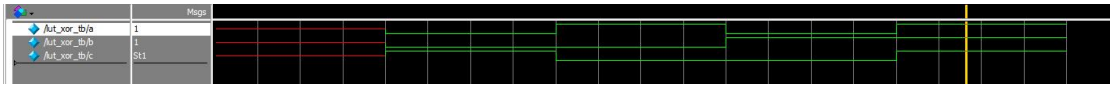




异或门电路 lut 结构化建模



同或门电路 lut 结构化建模

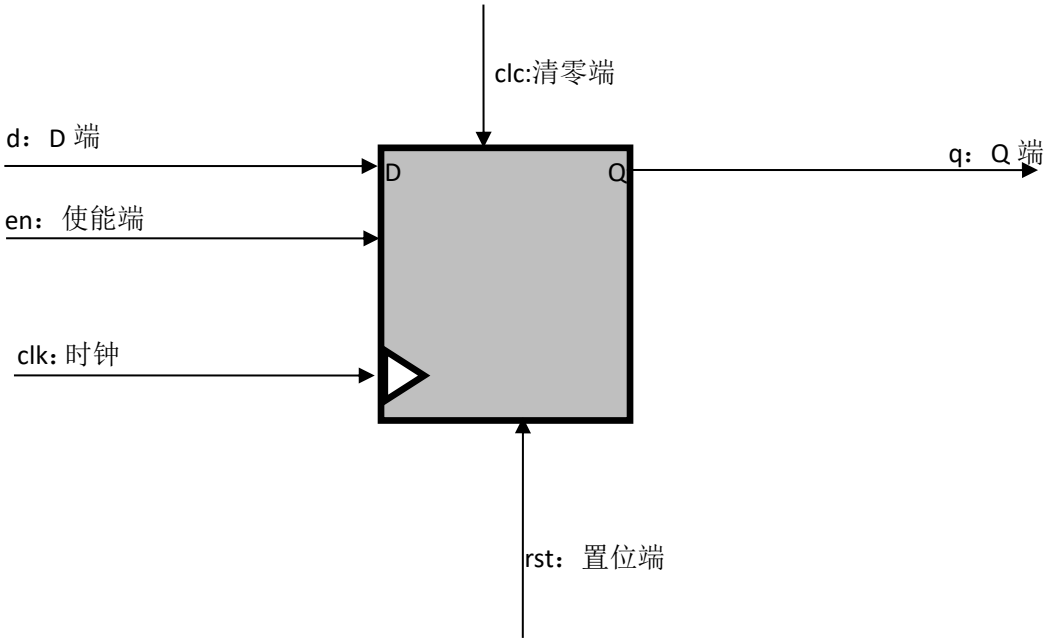


三：基础电路设计

对于数字电路设计，复杂的电路都是由简单的电力构成，在电路设计里相对于设计者而言有几种电路我们为基础电路，除触发器，多路器，译码器，分频器，计数器，对于这几种电路我们必须熟练掌握。

3.1 触发器

在 FPGA 种只用一种触发器，即 D 触发器，我们在 HDL 设计种必须正确使用 D 触发器。




```
always   @(posedge clk    or negedge    clc)
         if(clc)
              q<= 0;
         else if(en)
              q<=d;
```

```
always   @(posedge clk    or posedge    rst)
         if(clc)
              q<= 0;
         else
              q<=d;
```

D 触发器设计案例
标题：D 触发器设计案例
要求：
1：D 触发器带使能
2：D 触发器置位
3：仿真验证

3.2 多路器

多路器对于数字电路设计者而言位基础，本质是 HLD 语言种的 **if**...**else**..所描述的**控制电路**，最简单的多路器位 mux（单比特 2 选一多路器），在使用多路器的时候我遵循时许设计原则。

1：在 hdl 设计种如果被选择的控制通道较多的化，我们尽量转为并并行译码进行控制

```
if(条件)//通道 0
        ...;
else if(条件)//通道 1
        ...;
else    if(条件)//通道 2
    ...
  else if(通道...)//通道...
    ...;
```

NOTE:在 hDL 设计种，多路器和译码器可以相互转换。

2：多路器优先级的正确设定

在多路器设计种，优先级的设定结合被控制通道的电路的复杂都来设定。

如：4 通道多路器控制电路

```
if(sel[1:0]==2'b00)//第 0 通道
        y=a+b;//第 0 通道被控制的电路
else if(sel[1:0]==2'b01)//第 1 通道
        y=a*b;//第 1 通道被控制的电路
else if(sel[1:0]==2'b10)//第 2 通道
        y=a+b-3;//第 2 通道被控制的电路
else if(sel[1:0]==2'b11)//第 3 通道
        y=a*b+5;//第 3 通道被控制的电路
```

上述电路设计被多路器控制的个通道控制的复杂度排名以此是：第 3 通道>第 1 通道>第 2 通道>第 0 通道,正确的优先级设定如下;

```
if(sel[1:0]==2'b00)//第 0 通道
        y=a+b;//第 0 通道被控制的电路
else if(sel[1:0]==2'b01)//第 2 通道
        y=a+b-3   ;//第 2 通道被控制的电路
else if(sel[1:0]==2'b10)//第 1 通道
        y=a*b;//第 1 通道被控制的电路
else if(sel[1:0]==2'b11)//第 3 通道
        y=a*b+5;//第 3 通道被控制的电路
```

3.3 译码器

译码器对于 HDL 设计者而言位基础电路，我们必须熟练正确掌握译码器的设计，它是一种控制电路，在 HDL 种用 **case** 控制语句构建。

在 HDL 设计中译码器的设计，根据所采用的设计方式和设计目标不同，有 3 类译码器。

译码器设计类型	特点	HDL 控制语句	例子

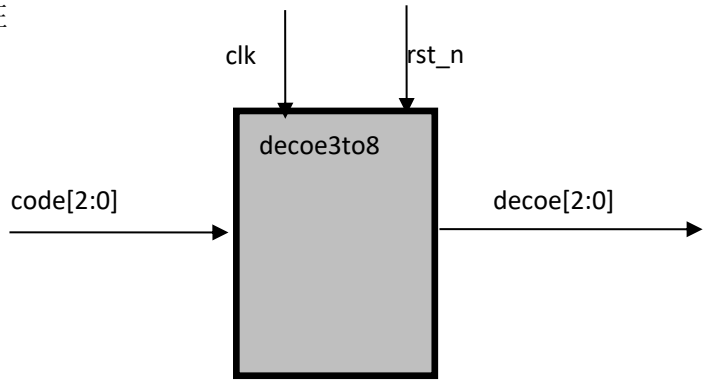
并行译码器	1：时许特性较好
	2：没有优先判决电路
		并行 case	case (sel)
			2'b00: out = a; // 互斥分支 1
			2'b01: out = b; // 互斥分支 2
			2'b10: out = c; // 互斥分支 3
			2'b11: out = d; // 互斥分支 4 // 无重叠，综合为 4 选 1 MUX（并行选择）
			endcase

全译码器	1：包含了所有译码的可能
	2：比较稳定
		full case	case (sel)
			2'b00: out = a;
			2'b01: out = b;
			default: out = 4'b0000; // 覆盖 sel=2'b10/11 的情况
			endcase

普通译码器	1:有优先判决电路
	2：时许特性一般
	3：没有要求全译码
		Case	case (sel)
			2'b00: out = a; // 分支 1
			2'b01: out = b; // 分支 2
			2'b00: out = c; // 重叠分支（永远不会执行）
			// 遗漏 sel=2'b10、2'b11 的情况，且无 default
			endcase

在 HDl 语言中并没有针对译码器的设计类型进行定义，针对这种情况，很多综合器支持通过指令指示综合器，我们在使用的时候先用先查。

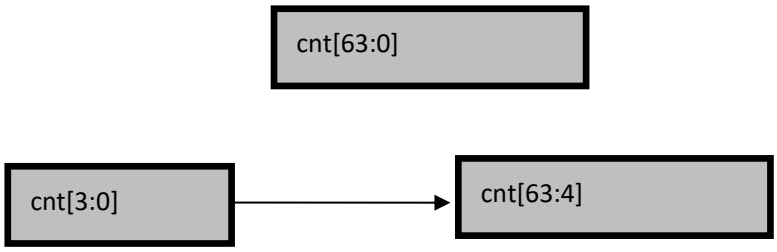
三八译码器设计案例
标题：三八译码器设计
要求：
1：全译码
2：用时许电路实现
3：仿真验证



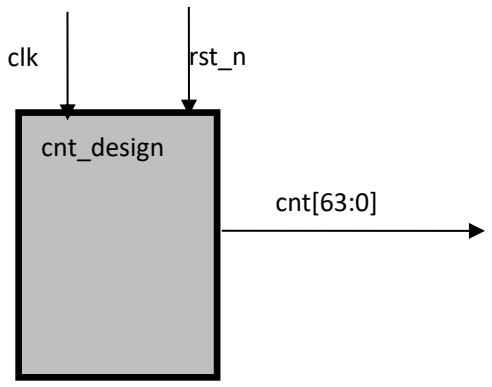
3.4 计数器

计数器对于 HDL 设计者而言位基础电路，我们必须熟练正确掌握译码器的设计，它是一种控制电路。

预定标技术：对于高位计数器，如 cnt[63:0],这种计数器在复杂电路中应用，为了让设计时许和电路更稳定，通常采用预定标技术。



计数器设计案例
标题：计数器设计
要求：
1：64 位计数
2：采用预定标计数
3：仿真验证


clk
rst_n
cnt_design
cnt[63:0]